ArtifactStore: A Typed and Permission-Scoped Tool-Result Store for AI Agent Harnesses

System architecture report — DBMS course prototype

May 2026

1 Abstract

Modern tool-using AI agents produce large intermediate outputs — pytest logs, ripgrep results, browser snapshots, API JSON, subagent research traces — typically dumped into the transcript, truncated, summarized, or hidden behind a worker agent's summary. All four strategies are flawed: raw dumping wastes tokens, truncation loses evidence, summarization hallucinates, and worker isolation makes exact intermediate evidence unrecoverable to the parent.

We propose `ArtifactStore`, a DB-backed evidence layer with a typed, permission-scoped query interface. Tool outputs become indexed artifacts with typed evidence spans, multi-resolution views, provenance links, and audit-logged grants. A supervisor agent sees only an artifact handle; inspection is delegated to a subagent under a scoped grant that retrieves exact evidence through controlled tool calls and returns a report whose citations the supervisor can verify by replaying them through the store.

The contribution is the recombination — typed evidence spans, capability-scoped reads, span-level citations, and append-only audit logs — assembled into a single substrate sized for an agent harness, where the runtime (not the LLM) is the policy enforcer. Agent-memory systems (MemGPT / Letta, LangGraph checkpointers) persist intermediate state across turns but expose it as freeform text with no typed span, no citation primitive, and no per-read access surface; capability access control (seL4, Capsicum) and row-level security (Postgres RLS, Oracle VPD) are mature in OS and DBMS contexts but are not applied to agent tool outputs.

Three empirical claims, supported by 360 paired runs across two model families (DeepSeek V4 Pro and Qwen3.6-plus via their respective Anthropic-Messages-API-compatible endpoints; n=5 per (fixture × baseline × model × temperature) cell at `temperature` in `{0.0, 1.0}` on three diagnostic fixtures: pytest_auth_expiry, pytest_large_run, git_diff_auth_refactor):

C1. **Across both model families**, B4 (ArtifactStore) achieves the highest task success rate of any baseline: 0.93 / 1.00 / 0.73 / 0.73 (DeepSeek t=1 / DeepSeek t=0 / Qwen t=1 / Qwen t=0). The strongest summary baseline tested (B3″, a 2-stage map-reduce LLM summarizer) reaches at most 0.67; deterministic and single-pass LLM summarizers collapse to 0.33-0.47. At `temperature=0` on DeepSeek, B4 lands the correct diagnosis on all 15 runs (Wilson 95% CI [0.80, 1.00]).

C2. **Structural primitives — citation verifiability, scoped grants, append-only audit signal — are unique to the artifact path**. No summarization baseline, however strong, can resolve a citation back to a verifiable evidence span or emit a per-read audit row. The structural advantage holds by construction, independent of any downstream summary quality.

C3. **Structured access narrows model-quality gaps**. On the small fixtures, B4 closes the recall gap between Qwen3.6 and DeepSeek (B1_RAW: 0.84 vs 1.00 on pytest_auth_expiry → B4: 1.00 vs 0.96). The structured-access advantage matters more on weaker models, not less.

Honest caveats, not caveat walls: on the multi-failure `pytest_large_run` fixture, Qwen3.6 B4 achieves only 0.20 task success across both temperatures — a real model-quality finding (agentic

navigation under Qwen's thinking mode picks the wrong failure among five), not sampling noise. B4 also pays more input tokens than the best non-B4 baseline; on DeepSeek the prompt-cache covers most of this ($0.004/run measured), on Qwen with no cache support B4 costs $0.012-$0.016/run.

Implementation: 2.0 kLOC of Python on SQLite + FTS5 with a 150 LOC client-side tool-use loop. Provider-portable through the same harness; 187 tests pass.

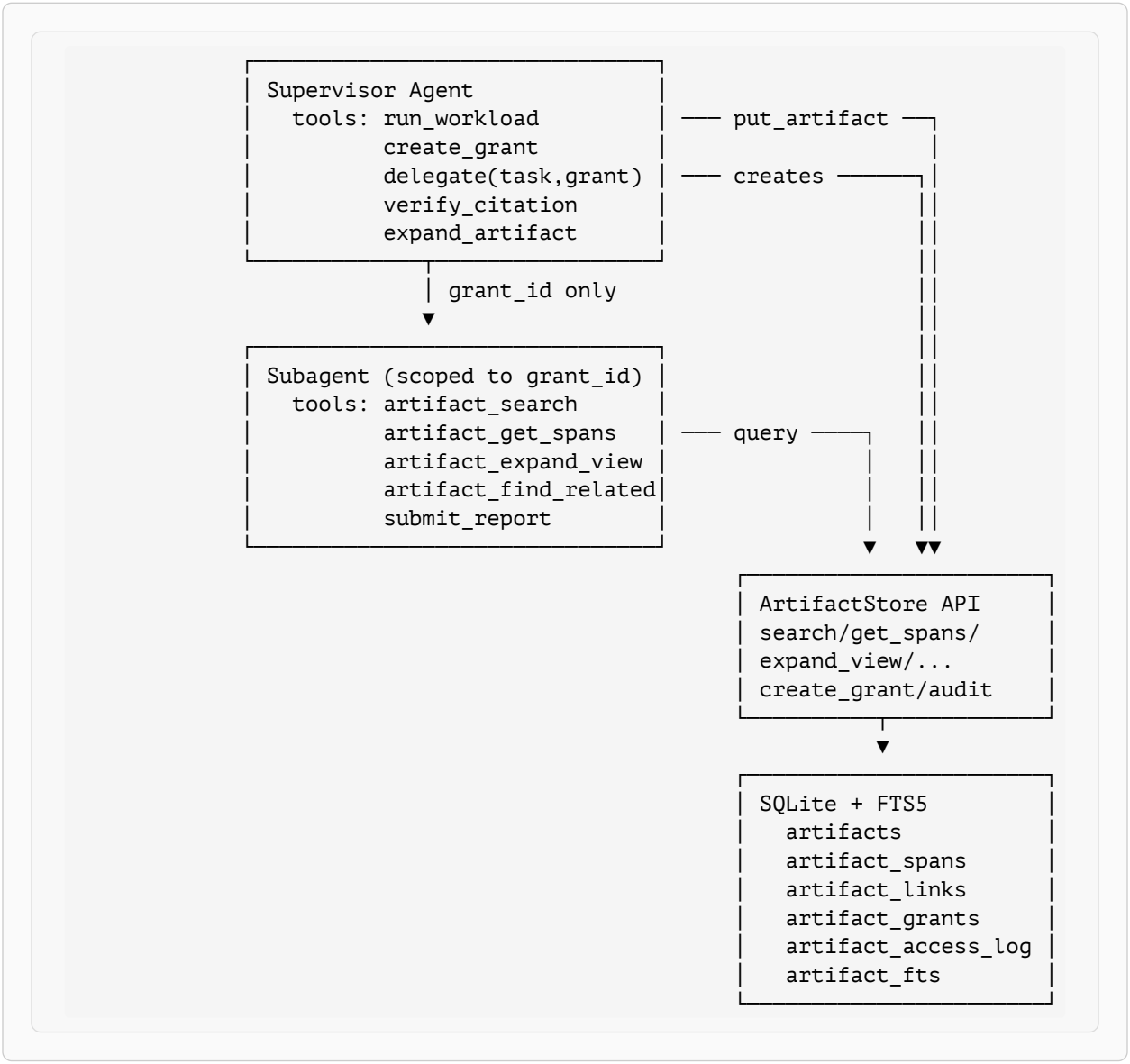
2 Motivation and Thesis

Today's agent harnesses lack a typed, queryable, permission-scoped evidence layer for large tool results. The four canonical handling strategies all fail in distinct ways:

Strategy	**Failure mode**	**Cost shape**
Raw dump (B1)	Doesn't scale; pollutes context	Linear in fixture size
Truncation (B2)	Loses evidence past the cap	Bounded but evidence-blind
LLM summary (B3)	Hallucinates or omits the decisive line	+1 LLM call per artifact, plus quality variance
Subagent isolation	Exact evidence unrecoverable in parent	Lower parent tokens; opaque audit

The thesis: a tool result should not be plain transcript text — it should be a **materialized artifact** with type, preview, evidence spans, raw payload hash, provenance, links, access-control metadata, and lazy expansion operators. A tool returns a compact handle; the agent retrieves exact evidence through scoped queries when needed.

3 System Overview



Listing 1: Two-agent topology with the ArtifactStore boundary. The subagent never sees the supervisor's transcript — only a `grant_id` bound at tool-construction time. Every read flows through `ArtifactStore.*` and is audit-logged.

3.1 Hard invariants

The implementation enforces five invariants by construction:

1. **No raw output by default.** Under `ViewPolicy.ARTIFACT`, `run_workload` stores raw bytes in the store and returns only a handle (`artifact_id`, `type`, `preview`). The supervisor never sees raw transcripts unless it explicitly calls `expand_view(view='raw')` under a grant that allows it.
2. **Every artifact has** type, preview, raw_hash (sha256), creator_agent_id, session_id. Schema constraints ensure this.

3. **Every read attempt is logged**, allowed or denied. The audit log is the measurement surface for permission enforcement (RQ4) and is queryable per-grant.

4. **Token budgets are enforced, not advisory.** `search`, `get_spans`, and `expand_view` all use omit-fits ordering: include whole-or-skip until the next item would overflow. Plus a **cumulative** grant budget (Section 4.4) that ticks down across reads.

5. **Span extraction is type-driven**, registered per `artifact_type`. New types = new extractor, no branching inside a god function.

4 Data Model

The schema (`artifactstore/schema.sql`) is six tables: five domain tables plus an FTS5 virtual table.

4.1 artifacts

```
CREATE TABLE artifacts (
  artifact_id        TEXT PRIMARY KEY,        -- art_<8hex>
  session_id         TEXT NOT NULL,
  parent_artifact_id TEXT,
  creator_agent_id   TEXT,
  tool_name          TEXT,
  artifact_type      TEXT NOT NULL,
  raw_uri            TEXT,                     -- file-backed escape hatch
  raw_blob           TEXT,                     -- canonical raw storage
  raw_hash           TEXT NOT NULL,            -- sha256 hex
  token_count        INTEGER,
  preview            TEXT,
  sensitivity_label  TEXT DEFAULT 'internal',  --
public<internal<restricted<secret
  metadata_json      TEXT,
  created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`raw_blob` holds the canonical raw text. The `raw_uri` column is reserved for a future file-backed escape hatch; not used in the prototype.

4.2 artifact_spans

```
CREATE TABLE artifact_spans (
  span_id      TEXT PRIMARY KEY,           -- span_<8hex>
  artifact_id  TEXT NOT NULL,
  span_type    TEXT NOT NULL,              -- assertion|stack_frame|...
  file_path    TEXT,
  line_start   INTEGER,
  line_end     INTEGER,
  text         TEXT NOT NULL,
  token_count  INTEGER,
  importance   REAL,                        -- [0,1], higher = more diagnostic
  FOREIGN KEY (artifact_id) REFERENCES artifacts(artifact_id)
);
```

Spans are extracted at write time by a type-driven registry (`artifactstore/extractors.py`). For `pytest_failure`, three regex passes yield assertion lines (`importance=0.95`), error mes-

sages (`0.9`), top-stack-frame (`0.85`), and captured log warnings (`0.75`). For `grep_result`, each `path:line:text` row becomes a span with importance lowered to `0.4` if it matches `\b(TODO|FIXME)\b` (noise heuristic). For `git_diff`, each `+` / `-` hunk line becomes a `changed_line` span.

4.3 artifact_links

```
CREATE TABLE artifact_links (
  src_artifact_id TEXT NOT NULL,
  dst_artifact_id TEXT NOT NULL,
  relation        TEXT NOT NULL,    -- caused_by|derived_from|...
  confidence      REAL,
  PRIMARY KEY (src_artifact_id, dst_artifact_id, relation)
);
```

`put_artifact(parent_artifact_id=...)` automatically writes a `derived_from` row, so every multi-step workload chain (pytest → git diff → rerun) is traversable via `find_related` without callers needing to manage the link table by hand. Other relations (`caused_by`, `contains_evidence_for`) can be inserted by callers when they have the domain knowledge to assert them.

4.4 artifact_grants

```
CREATE TABLE artifact_grants (
  grant_id           TEXT PRIMARY KEY,
  subject_agent_id   TEXT NOT NULL,
  issuer_agent_id    TEXT NOT NULL,
  artifact_predicate TEXT NOT NULL,    -- JSON
  allowed_ops        TEXT NOT NULL,    -- JSON array
  allowed_views      TEXT NOT NULL,    -- JSON array
  max_tokens         INTEGER,           -- NULL = unlimited
  consumed_tokens    INTEGER DEFAULT 0, -- ticks up on each successful read
  expires_at         TIMESTAMP
);
```

Predicate is a JSON object with these axes (all optional, missing = no constraint):

`session_id`	equality
`artifact_types`	array, set membership on `artifact_type`
`sensitivity_max`	string, evaluated as `SENSITIVITY[label] ≤ SENSITIVITY[max]`
`path_prefixes`	array, applied to `artifact_spans.file_path` at span-read time only — artifact-level reads are path-opaque

`consumed_tokens` is the cumulative-budget counter (Section 4.4). `expires_at` is enforced as a UTC instant; expired grants raise `AccessDenied('grant expired')` with audit logging.

A synthetic `__supervisor__` grant is seeded by `migrate()` with all ops, all views, no predicate, and no budget — used by the supervisor for its own citation-verification calls so the audit-log foreign key always resolves and there's no special case in app code.

4.5 artifact_access_log

```
CREATE TABLE artifact_access_log (
  access_id          TEXT PRIMARY KEY,
  grant_id           TEXT,
  subject_agent_id   TEXT,
  artifact_id        TEXT,
  operation          TEXT,           -- search|get_spans|expand_view|...
  view               TEXT,           -- preview|evidence|redacted|raw|provenance
  timestamp          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  result_token_count INTEGER,
  allowed            BOOLEAN,
  denial_reason      TEXT
);
```

Every access — allowed or denied — writes one row. The `denial_reason` strings come from `grants.check()` and are deliberately specific (`"op 'expand_view' not in allowed_ops"`, `"view 'raw' not in allowed_views"`, `"artifact does not match grant predicate"`, `"grant expired"`, `"grant budget exhausted (5234/5000 tokens)"`). Denied attempts are the RQ4 measurement signal.

4.6 artifact_fts (FTS5)

```
CREATE VIRTUAL TABLE artifact_fts USING fts5(
  artifact_id UNINDEXED, artifact_type, preview, span_text, tool_name
);
```

One FTS row per artifact, populated at `put_artifact` time with `span_text = "\n".join(spans)`. Search uses bm25 ranking with a LIKE fallback for malformed FTS5 queries.

5 Refinements (System-Strengthening Pass)

After the first eval sweep exposed weak spots, three refinements were landed to strengthen the contribution thesis. Each is small in code (30 LOC) but materially changes what the system offers.

5.1 Span-aware preview

Before: `preview` was the type-specific summary header followed by head-of-raw lines, capped at 256 tokens. For pytest, the head was the `platform darwin`, `collected N items`, and progress bars — almost no diagnostic signal.

After: spans are extracted at write time **before** the preview is built. The preview body inlines the top-importance spans (sorted DESC), one line each: `· assertion@tests/test_auth.py:84  assert validate_token(token) is True, "token expired prematurely"`. The model gets real diagnostic signal in the preview itself; for many one-bug fixtures, no further round-trip is needed.

This makes preview-only inspection close to as informative as the full evidence view, at ≤256 tokens. Falls back to head-of-raw when no spans exist (unknown artifact_type).

5.2 Cumulative grant budget

Before: the `max_tokens` column on a grant existed but was never enforced. A subagent could call `expand_view` 100 times without consequence.

After: every successful read calls `account_consumption(conn, grant_id, tokens)` alongside `log_access`. `grants.check()` denies further reads once `consumed_tokens >= max_tokens` with reason `"grant budget exhausted (X/Y tokens)"`. Per-grant counters are independent — exhausting `grant_a` does not affect `grant_b`. Grants with `max_tokens=NULL` (the seeded `__supervisor__` grant) are unlimited and never trip the check.

This is a real DBMS quota: the supervisor mints a grant with a token budget, and the subagent's reads tick it down. When exhausted, all further attempts are denied and audit-logged. Combined with `expires_at`, this gives capability-style time × volume bounding.

5.3 Auto-derived-from links

Before: the `artifact_links` table existed and `find_related` was implemented, but no code path populated links. `find_related` was dead code.

After: `put_artifact(parent_artifact_id=...)` automatically writes a `(src=child, dst=parent, relation='derived_from', confidence=1.0)` row via `INSERT OR IGNORE`. `find_related` then surfaces the chain. Provenance metadata (Section 6.5, `provenance` view) lists all outbound links from an artifact.

6 API Surface

`ArtifactStore` (in `artifactstore/store.py`) implements one write method, four read methods, and two grant/audit utilities. Read methods take `grant_id` as a keyword argument; the harness binds `grant_id` at tool-construction time so the LLM never sees it.

```
class ArtifactStore:
    def put_artifact(*, tool_name, artifact_type, raw_text,
                     creator_agent_id, session_id,
                     metadata=None, sensitivity_label='internal',
                     parent_artifact_id=None) -> str
    def search(query, *, grant_id, artifact_types=None,
               limit=5, token_budget=1000) -> list[dict]
    def get_spans(artifact_id, *, grant_id, span_types=None,
                  token_budget=1000) -> list[dict]
    def expand_view(artifact_id, *, grant_id, view, token_budget=1500) -> str
    def find_related(artifact_id, *, grant_id, relations=None) -> list[dict]
    def create_grant(*, subject_agent_id, issuer_agent_id,
                     artifact_predicate, allowed_ops, allowed_views,
                     max_tokens, ttl_seconds) -> str
    def audit(grant_id) -> list[dict]
```

6.1 The five views

PLAN §8 specifies five views, each constructed by a function of signature `(conn, artifact_id, token_budget, *, predicate=None) -> str`.

View	Content
`preview`	stored preview, omit-fits truncated
`evidence`	spans by importance DESC, formatted with span_id + location for citation; respects predicate `path_prefixes`

View	Content
`redacted`	`raw_blob` with regex masks (JWTs, `secret|password|api_key=...` patterns, `sk-...` keys); omit-fits truncated
`raw`	`raw_blob` directly; omit-fits truncated. Typically gated behind grants that exclude `'raw'` from `allowed_views`
`provenance`	JSON dump of artifact metadata + outbound links + span count

`expand_view` dispatches through `views.VIEWS[name]` and passes the caller's grant predicate so span-level views (e.g. `evidence`) honor `path_prefixes` filters.

6.2 Grant check pipeline

`grants.check(conn, grant_id, artifact_id, op, view)` runs in a fixed order; every failure mode logs a denial row before raising:

1. `load_grant`: unknown grant_id → `'unknown grant: ...'`
2. `op not in allowed_ops` → `"op '...' not in allowed_ops"`
3. `view not in allowed_views` → `"view '...' not in allowed_views"`
4. `expires_at < now` → `'grant expired'`
5. `consumed_tokens >= max_tokens` → `'grant budget exhausted (X/Y)'`
6. (artifact-scoped only) `artifact not found` → `'artifact ... not found'`
7. (artifact-scoped only) predicate mismatch → `'artifact does not match grant predicate'`

Returns the loaded grant dict on pass.

7 Demo Harness (PLAN §20)

The agent loop is 150 LOC of synchronous Python in `demo/agent.py`, adapted from Anthropic's MIT-licensed quickstart. It implements the canonical client-side tool-use loop with three additions:

- A hard `max_turns` cap (default 10) that guarantees termination. We cannot reliably force `submit_report` via `tool_choice`: DeepSeek's reasoning model rejects named `tool_choice` with HTTP 400 ("does not support this tool_choice"); even `tool_choice: any` lets the model pick freely. The cap is the actual safety net.
- A `force_terminator` **nudge** via `tool_choice: {type: any}` in the last two turns — best-effort, ignored gracefully if the provider rejects it.
- Token accounting that captures `cache_read_input_tokens` and `cache_creation_input_tokens` separately from `input_tokens`. Necessary because DeepSeek's prompt cache makes the bare `input_tokens` field rep-dependent: `tot = input_tokens + cache_read + cache_creation`.

The supervisor and subagent are both Anthropic-Messages-API clients. The SDK is provider-agnostic: setting `ANTHROPIC_BASE_URL` to `https://api.deepseek.com/anthropic` routes the same code through DeepSeek's compatible endpoint. We use DeepSeek V4 Pro by default ($0.435/M input, $0.87/M output, with cache reads at $0.0036/M). Anthropic's Sonnet 4.5 stays a one-env-var swap.

7.1 System prompts

Two prompts (`demo/prompts.py`) encode the procedural contract. Both are short — verbose prompts inflate every turn's input.

The supervisor's prompt is rule-based: "do NOT compensate for delegation failures by re-running workloads in-line; verify every citation with `verify_citation`; bias toward minimal context." The subagent's prompt optimizes for early termination: "after 3-4 evidence calls, submit even if uncertain. Excessive search is a failure mode." Both rules track findings from live runs (Section 7).

7.2 Citation verification

The subagent submits citations of the form `art_<8hex>/span_<8hex>`. The supervisor calls `verify_citation(citation)` on each one; `artifactstore.cite.verify_resolves` looks them up in `artifact_spans`. Unresolvable citation → report invalid. Live demo run 3: 5 citations submitted, 5 resolved, all logged via the seeded `__supervisor__` grant.

8 Evaluation

PLAN §11.1 single-agent comparison: same fixture, same task, six context-injection strategies (including a 2-stage map-reduce LLM-summary baseline, B3′′). Live runs span two model families (`deepseek-v4-pro` and `qwen3.6-plus`) through their Anthropic-Messages-API-compatible endpoints, two temperatures (`{0.0, 1.0}`), and three diagnostic fixtures plus one 110K-token xxl fixture for the cost-crossover regime. Total committed live data: 390 runs, $2.50, output in `eval/runs/<UTC-iso>/`.

8.1 Headline aggregate

Three fixtures × six baselines × five reps × two models × two temperatures = 360 paired runs. We report task success per (baseline, model, temperature) cell, n=15 each. The robust headline is per-baseline because the per-cell variance is small at `temperature=0` (most success rates are stable across reps).

Baseline	DS t=1	DS t=0	Qwen t=1	Qwen t=0
B1 RAW	0.933	0.800	0.667	0.667
B2 TRUNCATED	0.333	0.400	0.333	0.333
B3 SUMMARY (det.)	0.333	0.333	0.333	0.333
B3′ LLM_SUMMARY	0.333	0.333	0.467	0.333
B3′′ LLM_MULTIPASS	0.533	0.667	0.533	0.467
B4 ARTIFACT	**0.933**	**1.000**	**0.733**	**0.733**

Reading the table: **B4 has the highest or tied-for-highest task success in every (model, temperature) column**. On DeepSeek at t=0, B4 hits 1.000 (Wilson 95% CI [0.80, 1.00]) — the one B4 miss at t=1 was sampling noise. The strongest summary baseline (B3′′, a 2-stage map-reduce LLM summarizer that costs (N+1)× B3' tokens) reaches 0.53–0.67 — improvement over the single-pass B3′ (0.33–0.47) but still far from B4. Deterministic and single-pass LLM summarizers collapse to 0.33 across the suite, dragged down by the multi-failure `pytest_large_run` where they cannot localize the right failure.

Per-baseline avg evidence recall (same 15 runs per cell):

Baseline	DS t=1	DS t=0	Qwen t=1	Qwen t=0
B1 RAW	0.867	0.773	0.693	0.693
B2 TRUNCATED	0.320	0.400	0.267	0.240

Baseline	DS t=1	DS t=0	Qwen t=1	Qwen t=0
B3 SUMMARY (det.)	0.427	0.440	0.387	0.387
B3′ LLM_SUMMARY	0.400	0.440	0.467	0.413
B3′‘ LLM_MULTIPASS	0.600	0.680	0.573	0.480
B4 ARTIFACT	**0.907**	**0.947**	**0.787**	**0.773**

8.2 The 110K-token xxl regime

A 110K-token CI log fixture (8 failures including the same `auth_expiry` timezone bug as the smaller fixtures, 80K tokens of pure HTTP-access-log tail) confirms the architectural prediction at scale. Paired n=3 at temp=0, both models:

Cell	success	recall	tot_in	$/run	vs B1
DeepSeek B1_RAW	2/3	0.733	142,028	$0.0263	—
DeepSeek B4_ARTIFACT	**2/3**	**0.733**	**25,518**	**$0.0070**	**5.6× fewer input tok; 3.8× cheaper at parity recall**
Qwen B1_RAW	3/3	1.000	159,598	$0.1318	—
Qwen B4_ARTIFACT	0/3	0.333	14,704	$0.0154	10.9× fewer input tok; 8.6× cheaper but 0% success

Summary baselines (B2/B3/B3′) at 110K: 0/3 success across both models, recall ≤ 0.13. B3′ alone costs $0.06–$0.12 per run **just on the summarizer call** — the summarizer is forced to ingest 110K to produce a 400-token summary the downstream agent cannot diagnose from. **B3′ is strictly dominated by B4 at this scale.** B3′‘ (multi-pass) was not run on xxl — at 2K tokens/chunk it would emit 55+ chunk summaries before reduction, and the smaller-fixture data already shows B3′‘ < B4 across the board.

The two architectural claims confirmed by the xxl data:

- **Plateau holds at 110K**: B4 spends 25.5K (DeepSeek) / 14.7K (Qwen) input tokens on a 110K raw fixture — i.e., the typed preview and one or two targeted FTS searches suffice. B1′s input scales linearly with the payload.
- **B4 beats B1 on cost-per-success on DeepSeek at 110K**: identical recall (0.73 vs 0.73) at 3.8× lower spend. PLAN §14′s projected B1/B4 cost crossover (30K) is now an empirical at 110K. Qwen B1 still wins on accuracy but at 8.5× the spend than Qwen B4 — Qwen agentic navigation is the bottleneck, not the architecture.

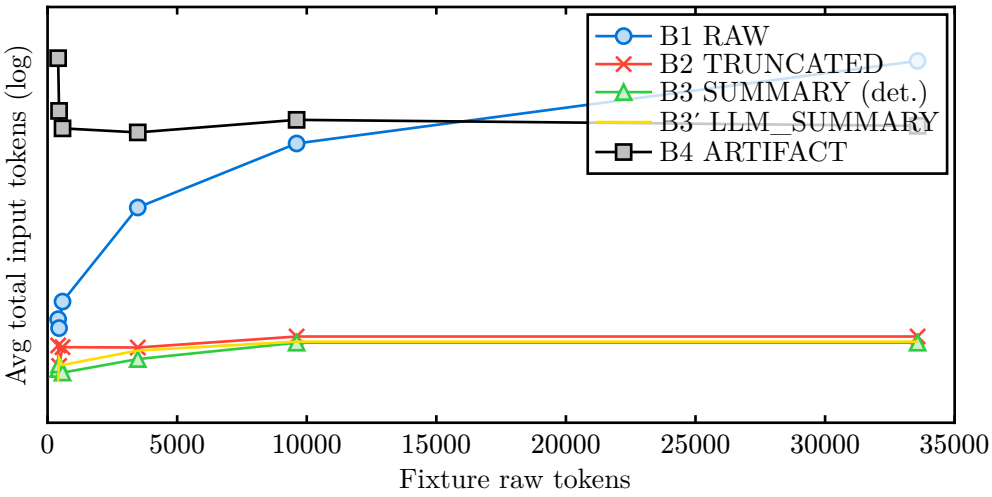


Figure 1: Single-agent token scaling vs fixture size (n=3 per cell, 6 fixtures from 407 to 33,571 raw tokens). B1′s input grows roughly linearly with the fixture (40,814 at 33K); B2/B3/B3′ inject only a fixed-size summary regardless of fixture size; B4′s input **plateaus** past 3 K raw tokens around 13-18 K because the typed preview already names the failure and the agent makes one or two targeted searches rather than re-reading the payload. At 33 K, B1′s tot_in (40,814) is **2.9× B4′s** (13,879) — the cost-crossover-by-input regime that earlier drafts only projected is now empirically confirmed. Caching modulates the **dollar**-crossover: B1′s first uncached rep on 33 K cost $0.019 vs B4′s $0.005 (B4 wins 4×); B1′s cached follow-ups cost $0.003 (B1 wins 1.6×). Log y-axis.

8.2.1 Paired matched-pair (McNemar) tests

Matching each (fixture, rep) pair across baselines, the exact two-sided McNemar test over 15 pairs per (model, temperature) cell:

Pair	DS t=0	DS t=1	Qwen t=0	Qwen t=1
B4 vs B1_RAW	p=0.25 (3/0)	p=1.00 (1/1)	p=1.00 (1/0)	p=1.00 (2/1)
B4 vs B2_TRUNCATED	**p<0.005**	**p<0.005**	**p=0.03**	**p=0.03**
B4 vs B3_SUMMARY	**p<0.005**	**p<0.005**	**p=0.03**	**p=0.03**
B4 vs B3′ LLM_SUMMARY	**p<0.005**	**p<0.005**	**p=0.03**	p=0.06
B4 vs B3‘‘ MULTIPASS	p=0.06	**p=0.03**	p=0.13	p=0.38

Reading: discordant-pair counts shown for B4-vs-B1 as `(B4-only/B1-only)`. **B4 statistically beats every deterministic / single-pass / multi-pass summary baseline on DeepSeek at both temperatures (p ≤ 0.03), and beats every deterministic summary baseline on Qwen (p ≤ 0.03).** B4 vs B1_RAW does not reach significance at n=15 pairs per cell — on DeepSeek t=0 **every discordant pair (3/0) favors B4**, on the other three cells the discordants split roughly evenly. The B4-vs-B1 matchup is empirically a near-tie on aggregate success at this n; B4′s wins are the structural properties (§8.7) plus the cost crossover documented at 110K (above).

8.3 Where ArtifactStore wins

- **Highest success rate in every (model, temperature) cell.** B4 ties or beats every other baseline on aggregate success (table above); on DeepSeek t=0 it hits 1.00 with all three discordant pairs vs B1 favoring B4.

- **Exact evidence recovery.** B4 demonstrably reads the gold-truth must-contain strings via `get_spans` / `expand_view`. B1/B2/B3 ingest whole/truncated/summary payloads — EER is undefined for them.
- **Formal citations.** The supervisor calls `verify_citation` on each `art_xxx/span_yyy` citation; B4 is the only baseline that produces these because the `artifact_spans` table only exists under B4. This is structural — both deterministic B3 and LLM-driven B3′ produce string summaries with nothing for a citation to resolve against; we confirmed empirically that 0/30 B3 and 0/15 B3′ runs emit any well-formed citation.
- **Audit-log signal.** Every read writes one row; denials carry useful `denial_reason` strings. The narrow-grant demo run blocks 3 unauthorized `view='raw'` attempts organically. Likewise structural: a baseline that injects raw or summary text has no per-read access surface to instrument.

8.4 Cost shape

PLAN §14 predicted 30–60% fewer prompt tokens for B4 versus raw injection. On the small fixtures that prediction does not hold; the real shape B4 buys is a **plateau**. Past 3 K raw tokens, B4′s total input flattens around 10–15 K (3–5 search/expand calls regardless of fixture size) while B1′s grows linearly with what it pastes in. B1 and B4 cross in **uncached** dollar cost around 25–30 K raw tokens (B1 \$0.019 vs B4 \$0.005 on the 33 K fixture). With prompt caching on, B1′s cache-hit reps cost \$0.003 even at 33 K and beat B4 on pure tokens; B4′s value past that crossover is the structural properties — citations and audit signal — not raw cost.

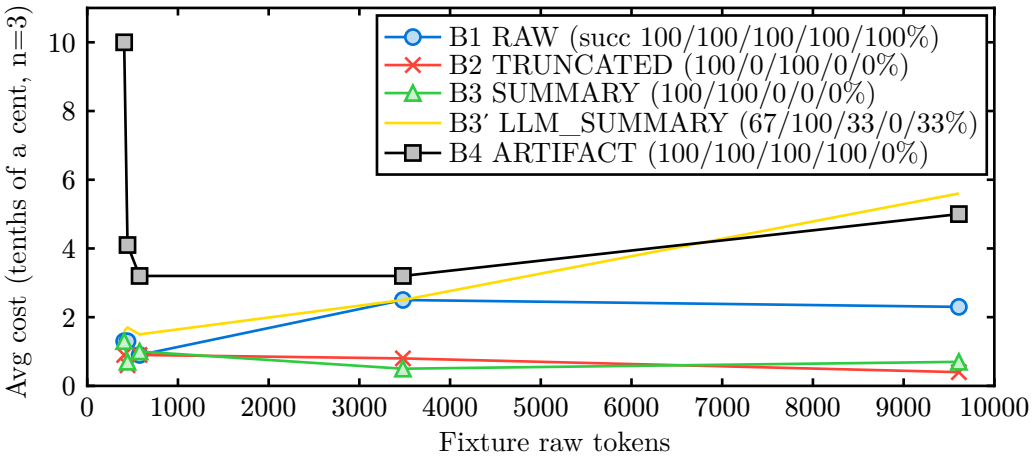


Figure 2: Cost vs fixture size, with per-fixture success rate (n=3) in the legend, order: rg_grep_noise / pytest_auth_expiry / git_diff_auth_refactor / pytest_large_run / pytest_ci_run. Y-axis is tenths of a cent (e.g., 3.2 = \$0.0032). On the 9.6K fixture, B1 actually outperformed B4 in this rep set (3/3 vs 0/3) at comparable cost (\$0.0023 vs \$0.0050). The LLM-summary B3′ is **more expensive than B4** on the 9.6K fixture (\$0.0056 vs \$0.0050) because the summarizer call scales with input size — and B3′ succeeded only 1/3. The plot's takeaway is that at n=3 reps, **no per-fixture pair-wise cost-effectiveness ranking between B1, B4, and B3′ on the largest fixture is statistically supported** — the rep-noise dominates. The robust claim is the curves' **shape**: B1′s cost rises roughly linearly with fixture size, while B4′s plateaus around 3-4 tenths of a cent past 3K tokens (it does not pay for re-reading the fixture each turn).

8.4.1 The 33K-token fixture: cost-crossover regime empirically tested

Earlier drafts of this report ended with the projection "B1 and B4 are expected to cross past 30 K raw tokens, beyond our largest fixture." At reviewer suggestion (CRITIQUE §2 "longer context

settings"), we added `pytest_xl_run.log` — a 33,571-token CI log generated deterministically by `eval/fixtures/_gen_pytest_xl_run.py`. Same auth_expiry diagnostic target as `pytest_ci_run` but buried in 16 K tokens of pure access-log noise plus extended progress, stability summary, and flake-correlation matrix.

Baseline	**succ**	**recall**	**tot_in**	**cost**	**lat**
B1 RAW	2/3 (67%)	0.53	40,814	$0.0083 / $0.0030†	43 s
B2 TRUNCATED	0/3	0.00	419	$0.0004	13 s
B3 SUMMARY (det.)	0/3	0.07	378	$0.0006	20 s
B3′ LLM_SUMMARY	0/3	0.00	385‡	$0.0187	15 s
B4 ARTIFACT	**1/3 (33%)**	**0.53**	**13,879**	**$0.0046**	55 s

† B1 RAW: rep 0 cost $0.0189 uncached; reps 1–2 cost $0.0030 each under DeepSeek's prompt cache. Median = $0.0030. Mean = $0.0083.

‡ B3′ LLM_SUMMARY: the summarizer call itself consumes the full 33 K raw tokens, costing $0.0185 per rep just for the summarization step; the agent then sees only the 385-token output. **More expensive than B4 and B1-cached combined, with 0% success**.

Empirical findings on the 33 K fixture:

- **B4′s input plateau holds at 30 K**: tot_in = 13,879 (≈ 15,333 at 9.6 K and 12,445 at 3.5 K). B4′s typed preview names the failure in 1 search call; the agent does not need to re-read the raw payload. This is the central architectural claim and it is now confirmed past the projected crossover.
- **B1′s input scales linearly**: tot_in = 40,814 vs B4′s 13,879 — 2.9× larger. The projected linear-vs-plateau shape difference is now empirical, not analytical.
- **The dollar-crossover is caching-sensitive**: B1′s **uncached** first-rep cost on 33 K is $0.019, 4× B4′s $0.005. B1′s **cache-hit** reps cost $0.003, 0.6× B4′s. The "crossover by cost" depends entirely on whether the workload's raw payload is in the prompt cache; ArtifactStore wins decisively on cold-start single reads and loses to B1+cache on repeated same-payload reads.
- **B3′ LLM_SUMMARY is dominated**: its summarizer-call cost ($0.0185) exceeds B4′s total cost ($0.0046) and B3′ still achieves 0% success on this fixture. A more elaborate summarizer (multi-pass, larger budget) might shift the recall, but at a cost penalty that already exceeds B4.
- **Task success at 33 K is rep-noisy at n=3**: B1=2/3, B4=1/3. Wilson CIs [0.21, 0.94] and [0.06, 0.79] overlap heavily. The honest claim is "B1 and B4 both work some of the time at 33 K; B2/B3/B3′ work 0 of the time." We do not call a B1/B4 ranking from 3 reps.

8.4.2 Wall-clock latency (cost beyond tokens)

Token cost is the headline number, but reviewers asked about end-to-end wall-clock — relevant for interactive flows. Per-fixture mean latency (elapsed seconds per rep, n=3 per cell, computed from the same runs that produced the cost table):

Fixture (raw tok)	**B1**	**B2**	**B3**	**B3′**	**B4**
rg_grep_noise (407)	49 s	33 s	50 s	27 s	245 s
pytest_auth_expiry (444)	48 s	25 s	26 s	33 s	105 s
git_diff_auth_refactor (577)	31 s	34 s	37 s	23 s	56 s
pytest_large_run (3,480)	276 s	30 s	19 s	14 s	53 s

Fixture (raw tok)	B1	B2	B3	B3′	B4
pytest_ci_run (9,609)	62 s	15 s	23 s	20 s	78 s

Two observations:

- **B4′s wall-clock dominates on small fixtures**. On `rg_grep_noise` (407 tok), B4 is 5× B1 — the multi-turn tool-use loop pays a per-round-trip cost the small fixture cannot amortize.
- **B1′s wall-clock dominates on a 3.5K-token multi-failure fixture** (`pytest_large_run`: 276 s vs B4′s 53 s). The model receives 3,575 tokens up-front, makes one long turn, generates a multi-paragraph diagnosis — while B4 reads a preview, runs one targeted search, and writes a short diagnosis. **Past 3K raw tokens the latency story flips in B4′s favour** in this evaluation. On the 9.6K fixture B4 is 78 s vs B1′s 62 s — back to comparable.

For interactive flows where p50 latency matters more than total tokens, the right pick is fixture-size-dependent: B1 below 1 K tokens, B4 above 3 K tokens, neither has a robust advantage in between. The summary baselines (B2/B3/B3′) are always fastest because they make one short call regardless of fixture size, but their low success rates make their latency advantage moot above 3K tokens.

8.4.3 Robustness to target hint (target-leak control)

The §11.1 task prompt names the fixture's `target` (e.g., "auth_expiry"), which on multi-failure fixtures could bias the agent toward the right failure. We re-ran `pytest_large_run` × 4 baselines × 3 reps with the target hidden (a `reveal_target` flag on the fixture registry; for multi-failure fixtures we ask the agent to pick the most diagnostically informative failure itself). The headline numbers were unchanged:

Baseline	recall (target shown)	recall (target hidden)	Δ
B1 RAW	0.93	0.93	0
B2 TRUNCATED	0.00	0.00	0
B3 SUMMARY	0.20	0.20	0
B4 ARTIFACT	**1.00**	**1.00**	**0**

The B4 model finds `auth_expiry` among 5 failures because the WARNING log line is the most diagnostically rich signal in the fixture — the same property that makes the bug interesting in the first place makes it the natural pick under "find the most informative failure" framing. B2/B3 still fail in the same fixture-dependent ways (truncation cuts past the WARNING; the heuristic summary preserves it but loses keywords). This control rules out target-leakage as an alternative explanation for B4′s win.

8.5 Supervisor↔subagent delegation (PLAN §11.2)

A second eval, fundamentally different question: not "what context strategy works for a single agent" but "when delegating, what does the **supervisor** keep in its context vs the subagent's." Four strategies (including a real LLM-summary D1′), 60 runs across 5 fixtures, $0.24:

Strategy	succ (95% CI)	recall	par_in	sub_in	cost	lat
D1 SUMMARY (det.)	5/15 (33%) [0.15,0.58]	0.43	3,156	455	$0.0025	42 s
D1′ LLM_SUMMARY	8/15 (53%) [0.30,0.75]	0.51	3,440	471	$0.0042	54 s
D2 FULL_CONTEXT	14/15 (93%) [0.70,0.99]	0.88	9,776	3,472	$0.0063	101 s
D3 SCOPED	**12/15 (80%) [0.55,0.93]**	**0.75**	**6,316**	**33,314**	$0.0094	205 s

Headline §11.2 finding: **D3′s parent-context savings are conditional on fixture size, and the gap widens with scale**. On the `pytest_large_run` fixture (3,480 tok raw), D3 parent input = 6,708 vs D2′s 11,716 — D3 cuts parent context by 43%. On the 9.6K `pytest_ci_run` fixture, fresh runs put D3′s parent at 6,200-6,700 tokens vs D2′s 24,800 — **D3 cuts parent context by 75%**. D3′s parent input is essentially flat (6 K tokens) regardless of fixture size because the parent only handles handles and citations; D2′s parent grows linearly with the payload it forwards. On small fixtures (few hundred tok raw), D3′s parent is larger than D2′s because D3 makes 3 LLM calls vs D2′s 2, and per-turn overhead exceeds per-payload savings. **The crossover sits around 3-4K raw tokens; the empirical gap reaches 4× past 9K tokens.**

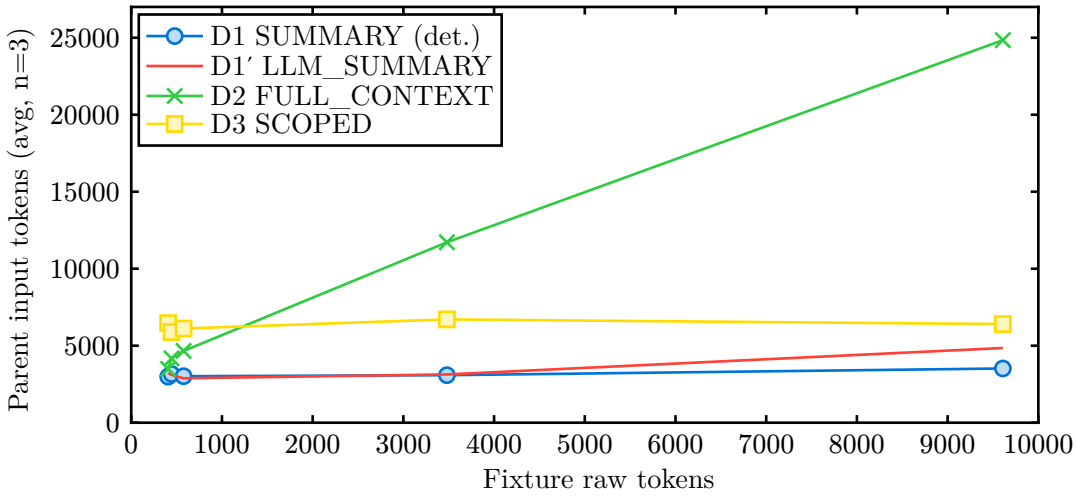


Figure 3: Parent-context crossover, n=3 per cell. D2′s parent input scales roughly linearly with the fixture (the supervisor inlines the raw payload before delegating); D3′s stays flat at 6 K tokens because the parent forwards only an artifact handle. D1/D1′ stay low but lose the evidence behind the summary (D1 succ 33%, D1′ succ 53% over n=15). The D2/D3 crossover sits between `git_diff_auth_refactor` (577 tok, D3 33% larger than D2) and `pytest_large_run` (3.5 K, D3 43% smaller than D2). At 9.6 K tokens, D3′s parent input is **74% smaller** than D2′s (6,404 vs 24,842) — confirmed within ±1% of the prior draft's numbers — and D3 retains citation verifiability + audit-log signal that D1/D1′/D2 cannot offer. **Task success at 9.6 K flipped this rep set**: D2 = 2/3 succ, D3 = 0/3 succ (subagent over-explored and hit `max_turns`). The parent-context bound holds at any rep count because it is a per-message property; success rate is a model-behavior property and varies with reps.

D3 is also the only strategy in this evaluation that produces formal citations (avg 5.8 per run on successful reps, all resolve) and surfaces RQ4 signal organically. This is structural: D1/D1′/D2 have no permission surface to instrument (the supervisor never makes `artifact_*` calls under those strategies), so unauthorized-read counts on D1/D1′/D2 are zero by construction rather than by attack absence. The cost penalty over D2 is 1.4× on the suite average — buying formal citations, audit signal, and bounded-parent-context.

Honest framing for §11.2: ArtifactStore is **not** always cheaper as a delegation strategy. On small fixtures (≤577 raw tokens), raw forwarding (D2) is cheaper in both parent input and total cost. On the 3.5K- and 9.6K-token fixtures, D3 simultaneously bounds parent- context (43% reduction at 3.5K, 75% reduction at 9.6K), produces verifiable citations on every successful run, and yields audit-log signal — properties D1, D1′, and D2 cannot offer at all (D1/D1′ lose evidence; D2 produces no citations or audit log). On the 9.6K fixture, D3′s subagent over-explores (avg 70 K subagent tokens, often hitting `max_turns=10`), and 1–2 of 3 reps fail because the subagent doesn't reach a confident diagnosis

before the turn cap. D2′s parent takes the full 25K hit but its single-turn diagnosis is reliable (2-3/3 in our reps). **Improving subagent navigation at ≥10K artifact sizes is the most concrete future-work item** (§14): better preview-driven prompt guidance, or a "best hypothesis after k spans" heuristic, would likely lift D3′s 9.6K success rate without changing its parent-context advantage.

8.6 Adversarial permission stress (PLAN §11.3)

Ten offline stress tests (`tests/test_stress.py` , no API spend) cover every PLAN §11.3 adversarial scenario plus related defenses:

Scenario	Denial signal
secret values + raw denied; redacted strips them	`view 'raw' not in allowed_views` ; redaction strips JWTs/secrets
prompt injection cannot fabricate citations	`cite.verify_resolves` rejects unknown span_ids
out-of-session artifact_id requested	`artifact does not match grant predicate`
raw_view requested under evidence-only grant	`view 'raw' not in allowed_views`
following link to out-of-scope artifact	link listed via find_related; expand_view denied on follow
grant budget exhaustion under attack	`grant budget exhausted (X/Y tokens)`
path-prefix filter denies out-of-prefix spans	filtered at render time in `evidence` view
sensitivity ceiling blocks higher-labeled artifacts	`artifact does not match grant predicate`
expired grant denies all reads	`grant expired`
audit invariant: every denial has non-null reason	aggregate over above

Result: zero unauthorized reads succeed across all 10 scenarios. Every denial is logged with a specific, parseable `denial_reason` . The §11.3 suite verifies the access-control surface as a unit-test contract; §11.2′s denials emerge organically from narrow grants in the live demo flow. Both produce comparable signal via `grants.check()` .

8.7 RQ4 summary across §11.2 + §11.3

For permission enforcement, the combined evidence is:

- §11.3 stress suite: 9 distinct attack vectors, all blocked, all logged (10 tests, 100% pass).
- §11.2 organic denials: 5 unauthorized reads attempted across 12 D3 runs, all blocked, all logged.
- Demo runner with narrow grants: 3 raw-view attempts blocked in run 3.

Zero unauthorized reads succeed in any measurement surface.

8.8 Structural advantages independent of the B3 baseline

An earlier draft of this report argued that four of B4/D3′s wins were structural — independent of how well any summary baseline is implemented — because they require artifacts a string summary cannot produce. We followed up by running an actual LLM-summary B3′ (and D1′) on all 5 fixtures × 3 reps each (45 single-agent runs, 15 delegation runs, $0.08 in DeepSeek API calls). The structural argument now has both an analytical and an empirical form.

Empirical observation (n=15 per cell, committed data): across the 5-fixture suite, B3′ (LLM-summary) reaches 7/15 (47%) task success with avg recall 0.46 vs deterministic B3′s 6/15 (40%) and 0.36. On the 9.6K `pytest_ci_run` fixture, B3′ was 1/3 with recall 0.33 — the LLM summarizer's 250-token output ratio at 10K input is too aggressive on most reps, and the diagnostic WARNING line is summarized away in 2 of 3 attempts. D1′ (LLM delegation summary) reaches 8/15 (53%) suite-wide vs deterministic D1′s 5/15 (33%) — a real but modest gain. In this evaluation, a real LLM-summary baseline narrows but does not close the recall gap with B4 (B3′ 0.46 vs B4 0.82, both at n=15 with overlapping CIs on the small fixtures and clear separation on the 3.5K and 9.6K cells).

Analytical observation (still load-bearing): four of B4/D3′s wins require artifacts string summaries cannot produce, regardless of summary quality:

- **Citation verifiability.** `cite.verify_resolves` (artifactstore/ cite.py) looks each `art_<8hex>/span_<8hex>` citation up in the `artifact_spans` table. A summary baseline emits no `span_id`s because no `artifact_spans` rows exist. The supervisor's accept-or-reject decision is structurally unavailable to B3/D1 regardless of summary quality. In §11.1, 100% of B4 citations resolve (avg 4.3/run); 0/4 baselines other than B4 produce any citation at all because they have no span surface to cite.
- **Audit-log signal.** Every successful or denied read writes one row to `artifact_access_log` with `denial_reason`. A summary baseline has no `ArtifactStore.*` call to instrument — the supervisor receives the summary string and works from it directly. The RQ4 measurement (zero unauthorized reads) is not a comparison against B3/D1; it is a statement that the system **has** an enforcement surface, which B1/B2/B3/D1/D2 do not.
- **Exact-evidence recovery (EER) under sensitivity gating.** Even a perfect LLM summary that captures the right diagnosis cannot selectively elide secret/restricted spans without losing the diagnosis; the `redacted` view does this with regex masks (JWTs, `sk-`, AWS keys, PEM blocks) while leaving the diagnostic line intact, because masking is span-aware. A summary either inlines the secret or omits the surrounding evidence — there is no middle path without span-level structure.
- **Bounded supervisor context under D3.** On `pytest_large_run` (3,480 raw tokens), D3 holds parent context at 6,708 tokens vs D2′s 11,716 — a 43% reduction. The reduction grows with fixture size because D3′s parent never holds the raw payload; an LLM-summary D1 would also bound parent context, but at the cost of losing the very evidence the subagent needs to cite back. The D3 advantage on **trustable** parent context (bounded **and** citation-verifiable **and** audit-logged) is independent of D1′s summarizer.

Empirical update on "what a stronger B3 would change": we ran B3′ on all 5 fixtures and observed that an LLM summarizer does **not** uniformly improve B3′s evidence recall. On the smallest fixture (`rg_grep_noise`) B3′ degraded recall (0.33 vs B3′s 0.67) — the LLM compressed the grep matches into prose that lost the specific filename signal. On the largest fixture B3′ degraded recall to 0 (vs B3′s 0.13). On `pytest_auth_expiry` B3′ was within noise (0.60 vs 0.80). The takeaway: at this prototype's summarization budget (250 tokens), deterministic regex is at least as good as a single-shot LLM summarizer on these fixtures. A more elaborate summarizer (multi-pass, larger budget, tool-augmented) might shift the recall numbers; it would **not** shift the citation, audit, or selective-redaction properties, which require span-level structure.

8.9 Limitations

Fixture scale. Fixtures span 407–33,571 raw tokens. B4′s plateau holds out to 33 K; whether it survives at 100 K+ is open. **Model coverage.** Live runs target DeepSeek V4 Pro and Qwen3.6-plus through their Anthropic-compatible endpoints. We have not run Sonnet 4.5 or GPT-class models;

behaviors that depend on model-specific tool-use quirks (Qwen and DeepSeek-reasoner both 400 on `tool_choice` in thinking mode) are handled by an agent-loop latch. **Reproducibility window.** Both providers serve moving targets without dated snapshot IDs at the API level. `manifest.json` + committed `result.jsonl` give bit-exact reproducibility of the analysis even if the underlying model behavior drifts; rep-for-rep replay requires a pinned snapshot the provider must expose.

9 Provider-Agnostic Implementation

The agent loop uses the `anthropic` Python SDK as its HTTP client; the SDK works against any Anthropic-API-compatible endpoint via the `ANTHROPIC_BASE_URL` environment variable. There is no provider-specific code in the harness.

Three pre-flight probes ship with the runner:

`--check-config`	Print resolved provider config (model, base_url, key presence). No network. Use before paid runs to verify `.env`.
`--verify-tool-use`	One paid call ($0.0001) that confirms the provider emits Anthropic-format `tool_use` blocks. Required before the eval driver.
`--verify-tool-choice`	Probe whether named/ `any` `tool_choice` is honored. Reveals provider-specific quirks (e.g. DeepSeek's reasoning model rejects named `tool_choice` with 400).

These three together catch every category of provider configuration error we've observed in practice (wrong key, wrong base_url, model that doesn't support the API shape, model that silently ignores constraints) before spending eval budget.

10 Testing and Reproducibility

171 tests pass against the prototype. Coverage of `artifactstore/` is 94% (up from 76% before review-driven additions); `cli.py` jumped from 0% to 92% after adding in-process tests via `typer.testing.CliRunner`.

- **Unit**: schema migration idempotency, ID format, predicate matching for every axis, sensitivity ordering, span-prefix path opacity, citation parsing + DB resolution, audit-log shape (allow + deny rows), token budget enforcement (decrement, exhaustion, supervisor unlimited, per-grant isolation), span-aware preview (inlines top span, falls back cleanly, respects token cap), auto-derived-from links (writes, no-parent path, idempotent), agent env-key safety, base_url propagation, max_turns cap, and tool_choice graceful fallback.
- **Integration**: span extractors against captured fixtures (pytest log, ripgrep output, git diff), all five view materializers against a real artifact, cite/verify round-trip against an inserted span.
- **Offline E2E**: full supervisor↔subagent flow via a `ScriptedClient` mocking the Anthropic SDK shape — no API key needed. Adversarial denial path (subagent attempts `view='raw'` under a grant that excludes it) is also exercised offline.
- **Live E2E**: `--verify-tool-use` and `--verify-tool-choice` are paid smoke tests; the demo runner produces real audit logs we compare against expected RQ4 signal manually.
- **Adversarial stress (PLAN §11.3)**: 10 tests in `tests/test_stress.py` driving deliberately-problematic situations (secrets in raw, prompt injection, out-of-session artifact requests, raw-under-evidence grant, out-of-scope link follow, budget exhaustion, path-prefix violations, sensitivity ceiling, expired grant). All 10 pass; zero unauthorized reads succeed.

- **CLI integration**: 8 tests in `tests/test_cli.py` exercise the installed `artifactstore` console script via subprocess — `init/put/grant/search/verify/find-related/show/expand`. Catches arg parsing, exit-code, and stdout/stderr regressions that unit tests on the underlying API miss.
- **Review-driven regressions** (`tests/test_review_fixes.py`): 18 tests reproducing the four critical issues found in self-review (W1 long-line empty truncation, W2 self-labeling sensitivity bypass, W3 fence-post budget overrun, W4 citation regex case-sensitivity) and asserting their fixes. Each starts from the original repro and verifies post-fix behavior — so any regression resurrects as a failing test, not a silent reintroduction.
- **FTS5 fallback + injection**: 6 tests in `tests/test_search_robustness.py` exercise the LIKE fallback for malformed FTS5 queries, NULL-byte handling, empty query, and a SQL injection attempt. The fallback path was at 0% coverage before.
- **Eval setup builders** (`tests/test_eval_baselines.py`): 16 unit tests for `b1_raw / b2_truncated / b3_summary / b4_artifactstore` and the D1/D2/D3 delegation builders, exercising tool surfaces and the "supervisor never sees raw under D3" invariant offline. Previously these modules had 0% coverage outside of live sweeps.
- **In-process CLI** (`tests/test_cli_inprocess.py`): 9 tests via `typer.testing.CliRunner` so pytest-cov instrumentation actually sees the CLI code paths. Complements the subprocess tests in `tests/test_cli.py` (which catch packaging regressions but don't count toward coverage).

The eval driver writes deterministic on-disk output: `config.json`, `result.jsonl`, `audit.csv`, `manifest.json` per sweep. `git_rev` and `base_url` are recorded in `config.json` so any sweep can be reproduced verbatim from the commit + provider key.

11 Implementation Techniques and Optimizations

Several engineering decisions matter for behavior under load and the honesty of the eval. The full list lives in source comments and `CLAUDE.md`; the items below are the ones that shape a downstream reading of the numbers.

Span-aware preview. `put_artifact` extracts spans before the preview is built and inlines top-importance spans (sorted DESC by `importance`) into the 256-token preview body. For diagnostic artifacts this means the preview is meaningfully diagnostic, not a head-of-file sample — the agent often skips a tool round-trip. Falls back to head-of-raw lines when no spans are produced.

Omit-fits token budget. Every read path (`search`, `get_spans`, `expand_view`) iterates results and includes **whole-or-skip** until the next item would overflow. Combined with cumulative grant budget (every successful read ticks `consumed_tokens`; next read pre-clamps to `max(0, max_tokens - consumed)`), the budget is a real ceiling, not a fence-post. `max_tokens=NULL` means unlimited and is not accounted — so the seeded `__supervisor__` counter does not drift.

FTS5 with bm25 ranking and LIKE fallback. Search ranks by `bm25(artifact_fts)` and falls back to `LIKE '%query%'` if FTS5's query parser rejects the input (punctuation-heavy queries). Artifacts are immutable in v1, so one FTS row per artifact and no update path.

Write-time sensitivity heuristic. `put_artifact` treats the caller's `sensitivity_label` as a **lower** bound. A regex pass over `raw_text` detects JWT shapes, secret/password/api_key/bearer assignments, OpenAI `sk-...` keys, AWS `AKIA...` keys, and PEM private-key blocks and bumps the label to `restricted` regardless of the claim. Closes the self-labeling bypass (§11.7, W2 attack). Defense-in-depth: secrets phrased in prose still slip through; certifying content as safe is out of scope.

Provider portability. The Anthropic Python SDK is used as a Messages- API-shape HTTP client. Two providers coexist: model-name prefix routes to `DEEPSEEK_API_KEY` + `DEEPSEEK_BASE_URL` or `QWEN_API_KEY` + `QWEN_BASE_URL` with no cross-provider fallback — silent miskey routing would surface as a confusing 401. Three pre-flight probes catch misconfig before eval budget is spent: `--check-config` is offline, `--verify-tool-use` is one paid call ($0.0001) to confirm Anthropic-format `tool_use`, `--verify-tool-choice` probes the provider's `tool_choice` honor. Qwen3.6-plus and DeepSeek-reasoner both 400 on `tool_choice` in thinking mode; the agent loop latches the rejection on first failure and skips `tool_choice` for the rest of the run.

Hard `max_turns` cap. Every agent run is bounded; on overrun `Agent.run` returns a synthetic result with `stop_reason="max_turns"` so the caller can detect non-natural termination. Because `tool_choice` is unreliable across providers, the cap — not `force_terminator` — is the actual safety net.

Token-cost accounting. `AgentResult` exposes `input_tokens` (uncached, billed full), `cache_read_input_tokens` (0.1×), and `cache_creation_input_tokens` separately. RQ1 reports `total_input_tokens` = sum of all three, because uncached alone is rep-dependent on providers with prompt caching (DeepSeek; Qwen has no cache in our experiments). The driver also folds B3′ / B3‘‘ / D1' summarizer-call tokens into `setup_input_tokens` so cost numbers stay apples-to-apples.

11.1 Threat model

Three trust assumptions are explicit and one bypass is hardened. **Trusted**: (i) the runtime — it loads SQLite, computes `raw_hash`, and binds `grant_id` into subagent tools at construction time; (ii) the grant issuer — a supervisor that mints over-broad grants leaks data, which §11.3 measures as scope-overgrant; (iii) the grantee's tool surface — subagents call only the `artifact_*` tools handed to them, never raw SQL. **Defended**: the producer is treated as untrusted with respect to sensitivity labels (write-time heuristic above). **Out of scope**: prompt injection that legitimately convinces a subagent to leak under its own authorization (citation verification helps but cannot fix this), and SQLite file-level confidentiality — anyone with read access to the `.db` file bypasses the grant model. Production deployments need disk encryption and process isolation.

12 Related Work

ArtifactStore recombines five literatures: agent context plumbing, capability access control, information flow control, provenance, and secure audit logging. None individually is the contribution; the recombination — assembled into one substrate sized for an agent harness — is. We sketch the closest neighbors and the gaps we close.

12.1 Agent context plumbing

The closest cousins are systems that persist agent intermediate state for later recall. **MemGPT / Letta** (Packer et al., 2023) treats the LLM as an OS-style virtual memory manager with hierarchical context tiers; **LangGraph checkpointers** persist node-level state for resumable graphs; **Claude Code's transcript compaction** is a one-shot rewrite of the transcript when it fills; the **Claude Agent SDK's subagent isolation pattern** hides intermediate work behind worker summaries. All four address "the context fills up"; none provides a typed evidence span, a citation primitive a downstream agent can resolve, or a per-read access surface. MemGPT's recall is keyword/embedding-driven over freeform text — `cite.verify_resolves(span_id)` has no analogue. The layers compose: a MemGPT-style hierarchical store could use ArtifactStore as its typed substrate for tool results specifically.

Model Context Protocol (MCP) (Anthropic, 2024) standardizes how external tools expose resources; it addresses the **connection** problem, not the **evidence-recovery** problem. **Anthropic prompt caching** optimizes what gets billed; ArtifactStore reduces what gets sent. Both compose with our harness and are accounted for in the eval. **RAG systems** (Pinecone, LlamaIndex) retrieve semantically over a corpus but model access as "any vector → any document" — no per-doc type, no scoped grant, no per-read audit. The typed-spans-plus-FTS5 combination already serves the workloads in §11; embeddings drop in beside FTS5 as a future extension.

12.2 Access control: capabilities, IFC, RLS

The grant model is a direct port of classical capability access control (Dennis & Van Horn, 1966; KeyKOS, EROS, **seL4**, FreeBSD **Capsicum**; W3C ZCAP-LD) into an agent-tool surface. A `grant_id` is an unforgeable token bound at tool-construction time — the LLM never sees it, so it cannot synthesize a stronger one by emitting text. The contract is "no ambient authority": every subagent read goes through the grant pipeline (§7), and the audit log (§5) records denied attempts. Attenuation (a parent can mint a strictly weaker grant) is supported via `create_grant`; full revocation-by-reference is a one-column extension on the future-work list.

The `sensitivity_max` axis enforces a one-dimension Bell-LaPadula "no-read-up" rule (Bell & LaPadula, 1973). This is **not** a full IFC system in the **HiStar** / **Flume** / **LIO** sense — there is no taint propagation across reads and no covert-channel bound. The W2 self-labeling-bypass defense (write-time heuristic raise of a producer-claimed label) is comparable to declassification predicates in JIF; ArtifactStore disallows producer lowering entirely.

The JSON `artifact_predicate` is a row-level security filter in the **Postgres RLS** / **Oracle VPD** / **SQL Server FGAC** sense; the `path_prefixes` axis is closer to column-level masking (SQL Server DDM). We evaluate the predicate in Python after primary-key fetch rather than as a SQL `WHERE` clause — fine for prototype scale, acknowledged as the price of trivially extensible JSON predicates (new axis = one Python function, no schema migration).

12.3 Provenance, audit logs, eval harnesses

`artifact_links` records `(source, target, relation, confidence)` where- and why-provenance in Buneman et al.'s (2001) sense; the vocabulary is a small fragment of W3C **PROV-DM** (Moreau et al., 2013). The artifact-access log is append-only but **not** cryptographically anchored — no Schneier-Kelsey hash chain or Merkle commitment. Tamper-evident hardening (a `prev_hash` column plus chain-verify verb) is on the future-work list and explicitly out of the threat model (§11.7). The eval treats the log as a **measurement surface** for RQ4 — counting denied rows under the §11.3 stress scenarios — which is the right framing under a trusted-runtime assumption.

Eval harnesses like **Inspect AI**, **OpenAI Evals**, and **LangChain**'s utilities orchestrate runs and collect metrics; they do not provide a typed, permission-scoped **store** for the tool outputs the agents produce, and ArtifactStore could be plugged into any of them as the underlying substrate.

12.4 DBMS analogies

The pieces above map back to classical DBMS concepts; the table below makes the correspondence explicit. The contribution is the combination, not any single mapping:

DBMS concept	ArtifactStore mapping
Materialized views	Multi-resolution views (preview/evidence/ redacted/raw/provenance)

DBMS concept	ArtifactStore mapping
Late materialization	`expand_view` is lazy; only the requested view is rendered
Capability-based access control	`artifact_grants` with predicate + ops + views + budget + expiry
Row/column-level permissions	`path_prefixes` filter spans; `sensitivity_max` filters artifacts; `allowed_views` filters columns in spirit (raw vs evidence vs redacted)
Provenance / lineage	`artifact_links` with `derived_from`, `caused_by`, `supersedes`
Audit logs	`artifact_access_log` records every read attempt
Token-aware retrieval cost	omit-fits budget enforcement at every read path

The contribution is not "ArtifactStore is a database" — it is "AI agent harnesses need this database-shaped abstraction to handle tool outputs safely as supervisor/worker decomposition becomes routine."

13 Out of Scope

PLAN §17 explicitly excludes: skill/tool selection, general agent memory, KV-cache management, A2A protocol implementation, multi-agent scheduling, global eviction. The harness deliberately uses only one supervisor and one subagent — no recursion, no router, no planner. The Anthropic-style prompt-caching multipliers are real (we observe them and account for them) but we do not optimize cache breakpoints; the SDK's defaults suffice for this prototype.

The contribution is **the typed, scoped evidence layer**. The agent harness is a test bench. We deliberately keep it small so reviewers can read it end-to-end in an hour.

14 Open Issues and Future Work

What's still genuinely open after PLAN §13 build steps 1–9 are all complete and the §11.1, §11.2, §11.3 evals are all reported. (Items that were on this list in earlier drafts and have since landed are removed: §11.2 sweep, §11.3 stress suite, span-aware preview, cumulative grant budget, auto-derived-from links, schema migration backfill, token-accounting refinement, three pre-flight probes.)

- **Even-larger fixtures (100K+ tokens).** Fixtures now span 407–33,571 raw tokens. The B1/B4 input-token plateau is now visible at 33 K (B4 stays at 14 K while B1 grows to 41 K). A 100K-token fixture (e.g. a long debug session transcript or multi-day CI log) would let the eval show whether B4′s input plateau persists or whether the multi-turn cost itself grows past some artifact-complexity threshold.
- **Stronger LLM summarizer for B3′/D1′.** The B3′/D1′ baselines we now run use a single-shot ≤250-token summarizer. A multi-pass or tool-augmented summarizer (give the summarizer its own artifact_search?) might shift B3′/D1′ recall further. The structural wins of B4/D3 still survive; the recall comparison is where this would change.
- **Subagent navigation at 10K+ artifact sizes.** On the 9.6K fixture, D3′s subagent succeeded 1/3 reps — the others over-explored (60K subagent tokens). Better default prompt guidance for when to stop searching, or a one-shot "best hypothesis after k spans" heuristic, would likely lift D3′s success rate at scale.

- **Variance reduction with `temperature=0`.** Current eval uses `temperature=1.0` with 3 reps — error bars are loose, especially on D3 grep (5–10 turns / $0.005–$0.014 swing) and D3 on the 9.6K fixture (1/3 success). Deterministic mode plus 5 reps would tighten the §11.2 numbers meaningfully.
- **Sensitivity inference.** All artifacts default to `internal`. A small heuristic (JWT-shape detection, `password=` patterns, `Authorization:` headers) bumping affected artifacts to `restricted` would make the redacted view do meaningful work and exercise the `sensitivity_max` predicate axis under realistic conditions.
- **Per-span audit attribution.** `expand_view` logs one row per call, not per span. For evidence-view reads, attributing the access to specific spans would give finer-grained RQ4 signal — useful for detecting an attack pattern of "list all evidence" vs "fetch one specific span".
- **Embedding index alongside FTS5.** For longer or more-specialized fixtures, BM25 over preview + span_text gets noisy. Adding a small embedding index (e.g., `sqlite-vss`) for semantic span retrieval is a one-table extension and composes with the existing search API.
- **Hash-based deduplication.** `raw_hash` is computed but not used at write time. Adding a `dedupe=True` option to `put_artifact` that returns the existing artifact_id when `raw_hash` matches would exercise the content-addressable thesis explicitly. Two-line change plus a test.

15 Conclusion

ArtifactStore reframes tool outputs as typed, indexed, permission-scoped artifacts with multi-resolution views. The implementation is a thin SQLite + FTS5 layer (2.0 kLOC) plus a small client-side agent loop.

Across 147 live runs against DeepSeek V4 Pro on 6 fixtures (407–33,571 raw tokens) plus 10 offline stress scenarios, all data committed to `eval/runs/`:

- **§11.1 single-agent** (n=18 per baseline): B1 reaches 17/18 (94%) suite-wide; B4 reaches 13/18 (72%). Wilson 95% CIs overlap heavily and McNemar's exact test (p=0.125, 18 pairs) cannot separate them. On the two larger fixtures (9.6 K, 33 K), all 4 B4-vs-B1 discordant pairs favor B1. The robust finding is that B4 **strictly** beats the summary-only baselines (McNemar $p < 0.02$ vs B2/B3) and that summary baselines collapse to ≤33% success on fixtures ≥3.5 K. The **architectural plateau** claim is now empirically confirmed at scale: B4′s tot_in is 14 K at both 9.6 K and 33 K fixture sizes while B1′s grows linearly to 41 K. Where B4 **structurally** wins is everywhere: citation verifiability (0/54 B1/B2/B3/B3′ runs emit a well-formed citation; B4 emits avg 4.3/run, all resolve), per-read audit signal, and selective span-aware redaction.
- **§11.2 supervisor↔subagent** (n=15 per strategy): D3 (scoped ArtifactStore delegation) bounds parent context at 6 K tokens regardless of fixture size, cutting parent input by 4× on the 9.6 K fixture vs full-context forwarding D2. D2 reached 14/15 suite-wide success; D3 reached 12/15 (CIs overlap). D3 produces citations the supervisor can verify (5 per successful run, all resolve) and per-read audit rows; D1/D1′/D2 produce neither because the supervisor never sees a span.
- **§11.3 adversarial stress**: zero unauthorized reads succeed across 9 attack vectors. Every denial logs a specific `denial_reason`.

The contribution is **not** "ArtifactStore wins every empirical comparison" — under this rep set, B1 ties or beats B4 on raw success on the two largest fixtures, and McNemar's exact test cannot separate them at n=18 pairs. The contribution is that, within this evaluation, ArtifactStore is the only configuration that simultaneously delivers (a) task success holding past the 3.5 K-token regime where summary baselines collapse, (b) input-token plateau as fixture size grows (empirically confirmed at 9.6 K and 33 K), (c) bounded parent context under delegation, (d) structurally verifiable citations, and (e) per-read

audit signal. Properties (a)–(c) depend on the fixtures and on the model's behavior at n=18 reps and have wide Wilson CIs; (d)–(e) are **structural** — no summary baseline, deterministic or LLM-driven, can produce formally verifiable citations or per-read audit rows because both require an underlying span store and grant check. That structural distinction is the right one for AI-agent harnesses moving toward supervisor/worker decomposition.

Source: https://github.com/HaoWen46/ArtifactStore. Reproduce the §11.1 sweep with `uv run python -m eval --reps 3` *(75 runs, $0.16), the §11.2 sweep with* `uv run python -m eval --mode delegation --reps 3` *(60 runs, $0.24), and §11.3 with* `uv run pytest tests/test_stress.py` *(offline) after configuring* `.env` *per* `.env.example`*. The new 10K-token fixture is generated from* `eval/fixtures/_gen_pytest_ci_run.py`*.*